Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on Laboratory Work No. 5

The File System with Your Own Hands

Operating Systems

Done by: st. gr. FAF-241, Cat Chiril

Checked by:
asist. univ. Reițman Patricia

Chișinău – 2026

Contents

Purpose and environment **1**

1 The directory tree and file types **1**

2 Inodes, links, and allocated space **1**

3 Permissions **3**

4 Disks and the kernel as files **4**

5 Handout questions and answers **5**

5.1 Part 1: the tree and file types 5

5.2 Part 2: inodes and links 5

5.3 Part 3: permissions 6

5.4 Part 4: disks and the kernel as files 7

Conclusion **7**

Purpose and environment

The goal was to inspect Linux path types, inode and link behavior, access permissions, disk layout, and kernel information exposed through `/proc`. I ran the experiments on 12 October 2026 as the normal user `class` in the installed Ubuntu guest under UTM, not as root. The commands ran in a new directory beneath `/home/class/os-lab5`. Their full output is preserved verbatim in the accompanying `lab5-results.txt`. The shorter command blocks below select and align relevant lines from that transcript. The CPU, RAM and disk figures describe this virtual machine.

1 The directory tree and file types

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx 1 root root 7 Apr 20 11:46 /bin -> usr/bin
crw-rw-rw- 1 root root 1, 3 Oct 12 16:11 /dev/null
drwxr-xr-x 142 root root 12288 Oct 12 16:11 /etc
-rw-r--r-- 1 root root 2769 Oct  5 15:58 /etc/passwd
drwxrwxrwt 16 root root 380 Oct 12 16:15 /tmp
$ ls -l /dev/vda
brw-rw---- 1 root disk 253, 0 Oct 12 16:11 /dev/vda
$ du -sh /var/log /usr
33M   /var/log
5.7G  /usr
```

Figure 1.1 – Recorded output for path types and disk usage

The leading `l` marks `/bin` as a symbolic link to `usr/bin`, so it resolves to `/usr/bin`. The leading `c` marks `/dev/null` as a character device, while `/dev/vda` starts with `b` and is a block device. The leading `d` marks `/etc` and `/tmp` as directories; the leading `-` marks `/etc/passwd` as a regular file. The `file` command also reported `/etc/passwd` as ASCII text and `/bin` as a symbolic link. `/usr` occupies about 5.7 GiB versus 33 MiB for `/var/log`: roughly 5.7 GiB more, or around 177 times as much. This reflects installed programs and libraries in `/usr` compared with current logs in `/var/log`.

2 Inodes, links, and allocated space

```
$ echo hello > notes.txt; stat notes.txt
Size: 6   Blocks: 8   IO Block: 4096   regular file
Inode: 1056330   Links: 1
$ ln notes.txt hard.txt; ls -li notes.txt hard.txt
1056330 -rw-rw-r-- 2 class class 6 Oct 12 16:15 hard.txt
1056330 -rw-rw-r-- 2 class class 6 Oct 12 16:15 notes.txt
$ rm notes.txt; cat hard.txt; ls -li hard.txt
```

```
hello
1056330 -rw-rw-r-- 1 class class 6 Oct 12 16:15 hard.txt
$ ln -s hard.txt soft.txt; ls -li hard.txt soft.txt
1056330 -rw-rw-r-- 1 class class 6 Oct 12 16:15 hard.txt
1061575 lrwxrwxrwx 1 class class 8 Oct 12 16:15 soft.txt -> hard.txt
$ rm hard.txt; cat soft.txt
cat: soft.txt: No such file or directory
$ stat soft.txt
Size: 8   Blocks: 0   symbolic link
```

Figure 2.1 – Recorded inode and link behavior

`Inode:` `1056330` identifies the file's inode. `Links:` `1` is its hard-link count: the number of directory entries naming that inode. Creating `hard.txt` raised the count to two and gave both names the same inode. `rm notes.txt` removed one directory entry, not the file data; `hard.txt` still named the inode and read `hello`. The symbolic link had its own inode, 1061575. Its contents were the eight bytes of the relative path `hard.txt`. After that target name was removed, the symlink remained but its path resolved nowhere, so `cat soft.txt` failed.

```
$ mkdir d; ls -ld d
drwxrwxr-x 2 class class 4096 Oct 12 16:15 d
$ mkdir d/x d/y; ls -ld d
drwxrwxr-x 4 class class 4096 Oct 12 16:15 d
$ echo x > one.txt; ls -l one.txt; du -h one.txt
-rw-rw-r-- 1 class class 2 Oct 12 16:15 one.txt
4.0K  one.txt
$ truncate -s 1G big.img; ls -lh big.img; du -h big.img
-rw-rw-r-- 1 class class 1.0G Oct 12 16:15 big.img
0  big.img
```

Figure 2.2 – Recorded directory links and sparse file allocation

The new directory's count of two comes from its entry in the parent and its own `.` entry. Each new child directory has a `..` entry that points back to `d`, raising `d`'s count to four. The two-byte file consumes one 4 KiB filesystem block. `truncate` made a sparse 1 GiB file: it set the logical length without writing blocks into the hole, so `du` reported zero allocated space. The test image was removed after measurement.

3 Permissions

Mode	Permission string	Typical use
600	`rw----`	Private file readable and writable by its owner
644	`rw-r-r-`	Publicly readable document, owner writable
755	`rwxr-xr-x`	Executable program or traversable directory
700	`rwx---`	Private directory accessible only to its owner

```
$ chmod 600 s.txt; ls -l s.txt
-rw------- 1 class class 7 Oct 12 16:15 s.txt
$ chmod 000 s.txt; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt; cat s.txt
secret
$ umask; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 class class 0 Oct 12 16:15 new.txt
drwxrwxr-x 2 class class 4096 Oct 12 16:15 newdir
```

Figure 3.1 – Recorded file permissions and umask

Ownership determines which permission class applies; it does not itself grant access. With mode 000, the owner's read bit was absent, so the normal user could not read `s.txt`. Restoring `u+r` made it readable. The umask `0002` removes the other-write bit from the default creation modes: `0666 & ~0002 = 0664` for the new file and `0777 & ~0002 = 0775` for the new directory.

```
$ chmod u=r,go= box; ls box; cat box/f
f
cat: box/f: Permission denied
$ chmod u=x,go= box; ls box; cat box/f
ls: cannot open directory 'box': Permission denied
hi
$ ls -l /usr/bin/passwd; ls -ld /tmp
-rwsr-xr-x 1 root root 142552 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 16 root root 380 Oct 12 16:15 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1311 Oct  5 16:10 /etc/shadow
```

Figure 3.2 – Recorded directory traversal and special permissions

For a directory, read permission allows listing entry names, whereas execute permission allows searching or traversing it by a known name. Thus read-only `box` listed `f` but blocked `box/f`; execute-only `box` could not be listed, but `cat box/f` succeeded because the file itself was readable. I restored `box` to mode 755 afterward.

The `s` in `passwd`'s owner-execute position is the set-user-ID bit: the program executes with root's effective user ID. It needs that privilege to update password data while ordinary users cannot read `/etc/shadow`. The `t` in `/tmp` is the sticky bit. Because `/tmp` is shared and world-writable, it restricts deletion and renaming of entries to their owner, the directory owner, or root.

4 Disks and the kernel as files

```
$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
vda       64G disk
|-vda1     1G part vfat     /boot/efi
‘-vda2  62.9G part ext4     /
$ df -h /
Filesystem  Size  Used  Avail  Use%  Mounted on
/dev/vda2    62G   12G    48G   20%  /
$ findmnt -no SOURCE,FSTYPE,TARGET /
/dev/vda2 ext4 /
$ grep -c ’^processor’ /proc/cpuinfo
6
$ grep ’^MemTotal:’ /proc/meminfo
MemTotal:        3461284 kB
```

Figure 4.1 – Recorded disk and memory observations

The virtual disk is `/dev/vda` (64 GiB). Its first partition, `/dev/vda1`, is a 1 GiB VFAT EFI partition mounted at `/boot/efi`. The 62.9 GiB second partition, `/dev/vda2`, contains the ext4 root filesystem. The root filesystem is 20% full, with 12 GiB used and 48 GiB available according to `df`. The VM exposes six logical processors and 3,461,284 kB of RAM, approximately 3.30 GiB.

```
$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   4301
Ngid:   0
$ ls -l /proc/self/fd
0 -> /dev/pts/0
1 -> pipe:[30151]
2 -> pipe:[30151]
3 -> /proc/4302/fd
```

Figure 4.2 – Recorded process status and file descriptors

`/proc/self` refers to the process that opens it. Here `head`, rather than the shell, opened `/proc/self/status`, so `Name` was `head`. Similarly, `ls -l /proc/self/fd` listed the file

descriptors of that `ls` process: standard input was the terminal, standard output and error were the transcript pipe, and descriptor 3 was its open fd directory.

5 Handout questions and answers

The questions below reproduce every required Observe prompt from the lab handout. The command output supporting each answer appears in the corresponding section above.

5.1 Part 1: the tree and file types

1. **Question:** For each path in the second line, write its type from the first letter of `ls -l`.

   Answer: `/etc` and `/tmp` are directories (`d`); `/etc/passwd` is a regular file (`-`); `/bin` is a symbolic link (`l`); and `/dev/null` is a character device (`c`).

2. **Question:** What is `/bin` on your system: a directory or a link, and to where?

   Answer: It is a symbolic link to `usr/bin`, resolving to `/usr/bin`.

3. **Question:** Find one character device and one block device in `/dev`.

   Answer: `/dev/null` is a character device; `/dev/vda` is a block device.

4. **Question:** Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much?

   Answer: In the measured Ubuntu guest, `/usr` was 5.7 GiB and `/var/log` was 33 MiB. `/usr` was roughly 5.67 GiB larger, or about 177 times as large.

5.2 Part 2: inodes and links

1. **Question:** Which number in `stat` is the inode, and which tells how many names the file has?

   Answer: `Inode: 1056330` is the inode number of `notes.txt`; `Links: 1` is the hard-link count, the number of directory entries naming that inode.

2. **Question:** After `rm notes.txt`, why could you still read `hard.txt`? What does `rm` really remove?

   Answer: Both names referred to inode 1056330. `rm` removed the `notes.txt` directory entry and reduced the link count from two to one. `hard.txt` still named the inode and its data, so `cat hard.txt` printed `hello`.

3. **Question:** Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?

   Answer: `soft.txt` stored the relative target path `hard.txt`. Once that target name was removed, resolution failed even though the symlink itself remained. Its size was 8 bytes, the length of the eight-character string `hard.txt`.

4. **Question:** A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.

   Answer: The new directory `d` is named once in its parent and once by its own `.` entry. Each child directory, `d/x` and `d/y`, has a `..` entry referring back to `d`, raising `d`'s link count to four.

5. **Question:** Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

   Answer: The two-byte file needs one allocated 4 KiB block. `truncate -s 1G big.img` set the logical length without writing data blocks; the file was sparse, so `du` reported zero allocated data space.

5.3 Part 3: permissions

1. **Question:** Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.

   Answer: 600 is `rw----` for a private owner-readable file; 644 is `rw-r-r-` for a publicly readable document; 755 is `rwxr-xr-x` for an executable or traversable directory; 700 is `rwx---` for a private directory.

2. **Question:** With `chmod 000` you own the file but cannot read it. Why does ownership not help?

   Answer: Ownership selects which class of mode bits applies; it does not override those bits. Mode 000 removes the owner's read permission, so the normal user receives `Permission denied`.

3. **Question:** What is your umask, and how does it explain the permissions of `new.txt` and `newdir`?

   Answer: The measured umask was `0002`. It removes other-write from default file mode 0666, giving 0664 (`rw-rw-r-`), and from default directory mode 0777, giving 0775 (`rwxrwxr-x`).

4. **Question:** Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?

   Answer: Directory read permission lets `ls` enumerate names; execute permission lets a process traverse a known name. With read only, `ls box` listed `f` but `cat box/f` failed. With execute only, listing failed but `cat box/f` printed `hi`.

5. **Question:** What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?

   Answer: `s` is set-user-ID: `passwd` runs with its owner root's effective privileges so it can update protected password data. `t` is the sticky bit: in shared `/tmp`, it restricts removal and renaming of entries to their owner, the directory owner, or root.

5.4 Part 4: disks and the kernel as files

1. **Question:** Draw your disk: disk name, partitions, which one is mounted at `/` and with what file system type.

   Answer: The 64 GiB virtual disk is `/dev/vda`. Its 1 GiB `/dev/vda1` partition is VFAT and mounted at `/boot/efi`. Its 62.9 GiB `/dev/vda2` partition is ext4 and mounted at `/`.

2. **Question:** How full is your root file system?

   Answer: `df -h /` reported 62 GiB total, 12 GiB used, 48 GiB available, and 20% use.

3. **Question:** How many CPU cores and how much RAM does your laptop have, according to `/proc`?

   Answer: The Ubuntu guest exposed six logical processors and `MemTotal: 3461284 kB`, approximately 3.30 GiB of RAM. These describe the guest VM, not the physical host.

4. **Question:** `/proc/self/status` showed the name of a program. Which one, and why that one?

   Answer: It showed `Name: head`. `/proc/self` refers to whichever process opens it; `head`, not the shell, opened the file in `head -5 /proc/self/status`.

Conclusion

The experiments showed that path names and file data are separate: hard links share an inode, while symbolic links store a path that can become dangling. Logical file length can differ greatly from allocated disk space. Permission checks depend on mode bits and directory traversal rights, with setuid and sticky bits serving specific shared-system needs. Finally, disk mounts and `/proc` exposed the guest's storage and runtime state through ordinary filesystem interfaces.